Planificación de CPU

Sistemas Operativos | Esteban Peña y Andrés Gámez | 4 de noviembre de 2026

Proyecto: simulación de gestión de procesos y comparación de FCFS con Round Robin (opción 1). Evidencias: cinco escenarios, diez ejecuciones comparadas y 31 pruebas automatizadas aprobadas.

1. Problema, objetivo y fundamento

Cuando varios procesos requieren una CPU, el orden de atención cambia su espera y su primera respuesta. El objetivo es representar esos estados, implementar dos políticas y comparar su comportamiento con cargas idénticas y resultados reproducibles.

FCFS atiende por orden de llegada hasta finalizar o bloquearse. Round Robin (RR) limita cada turno con un quantum y devuelve al final de la cola al proceso que aún necesita CPU [1]. Un proceso bloqueado espera E/S y vuelve a LISTO cuando termina esa espera [2].

2. Diseño y escenarios

Un motor de eventos compartido usa una cola FIFO de listos y una cola ordenada de llegadas y fin de E/S. models.py valida datos; algorithms.py ejecuta las políticas; scenarios.py define las cargas. app_visual.py conecta HTML, JavaScript y CSS con el motor y guarda resultados. main.py exporta CSV. Las funciones están documentadas y cada política recibe copias nuevas de los procesos.

Caso / propósito	Procesos: PID (llegada, CPU)	Q RR
E1. Carga escalonada	P1 (0,5); P2 (1,3); P3 (2,8); P4 (3,6)	3
E2. Efecto convoy	P1 (0,12); P2 (1,2); P3 (2,4)	3
E3. Llegadas simultáneas	P1 (0,4); P2 (0,6); P3 (0,8)	4
E4. Bloqueo por E/S	P1 (0,6), E/S de 4 tras consumir 2 CPU; P2 (1,5); P3 (2,3)	2
E5. CPU ociosa	P1 (3,2); P2 (10,3)	2

3. Métricas y reglas del modelo

Respuesta = primer inicio de CPU - llegada. Retorno = fin - llegada. Espera = retorno - CPU - bloqueo; también se acumula en la cola de listos. Las medias son promedios por proceso. Utilización = CPU ocupada / último fin, contando desde t=0.

Cambio de contexto: tramos contiguos de procesos distintos, excluyendo ocio y primer despacho. Despacho: cada asignación de CPU. Estados: NUEVO, LISTO, EJECUCIÓN, BLOQUEADO y FINALIZADO. A igual instante se admiten llegadas y desbloqueos antes de reinsertar al que agotó quantum; las llegadas originales preceden a desbloqueos. Los empates conservan orden de registro.

4. Resultados de las ejecuciones nuevas

Fuente: evidencias/ejecucion/resumen.csv, contrastada con los cinco JSON de la aplicación. Tiempos en unidades simuladas; CC = cambios de contexto. Valores redondeados a dos decimales.

Caso	Política	Espera	Respuesta	Retorno	CC	CPU %
E1	FCFS	5.75	5.75	11.25	3	100.00
E1	RR Q=3	8.50	3.00	14.00	7	100.00
E2	FCFS	7.67	7.67	13.67	2	100.00
E2	RR Q=3	4.67	1.67	10.67	5	100.00
E3	FCFS	4.67	4.67	10.67	2	100.00
E3	RR Q=4	6.00	4.00	12.00	4	100.00
E4	FCFS	3.33	2.00	9.33	3	100.00
E4	RR Q=2	5.33	1.00	11.33	7	100.00
E5	FCFS	0.00	0.00	2.50	0	38.46
E5	RR Q=2	0.00	0.00	2.50	0	38.46

Interpretación y sensibilidad al quantum

E2: P2 llega en 1. FCFS lo inicia en 12 (espera 11); RR Q=3 lo inicia en 3 (espera 2). Sin embargo, E1, E3 y E4 muestran mayor espera y retorno medios con RR. RR mejora la respuesta media en cuatro escenarios; en E5 empatan. No existe un ganador universal.

E2 / quantum RR	Espera media	Respuesta media	Retorno medio	CC
1	4.33	0.33	10.33	11
3	4.67	1.67	10.67	5
6	5.67	3.67	11.67	3
12	7.67	7.67	13.67	2

Un quantum menor produce más turnos y cambios, no más trabajo de CPU. En E2, Q=12 reproduce las métricas de FCFS. El costo temporal de cambiar de proceso es cero en este modelo, por lo que los conteos no son sobrecarga medida.

Equidad, inanición y estados

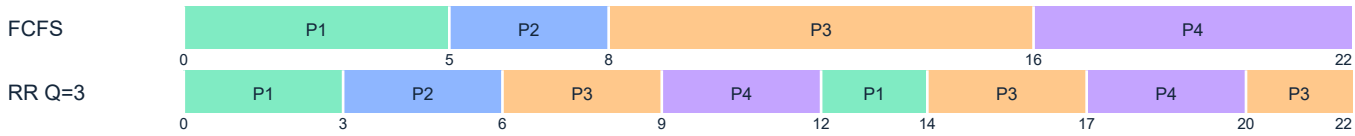
El máximo de respuesta de E2 baja de 12 con FCFS a 3 con RR, señal de acceso inicial más repartido en esa carga. Todos los procesos de las pruebas finitas terminan: no se observó inanición. No se estudiaron cargas infinitas ni se propone un índice universal de equidad.

En E4, P1 ejecuta 0-2, queda bloqueado hasta 6 y vuelve a LISTO en 6. FCFS lo reanuda en 10 y RR en 8. En E5, los tramos 0-3 y 5-10 son ociosos: CPU ocupada 5 de 13 unidades (38,46 %).

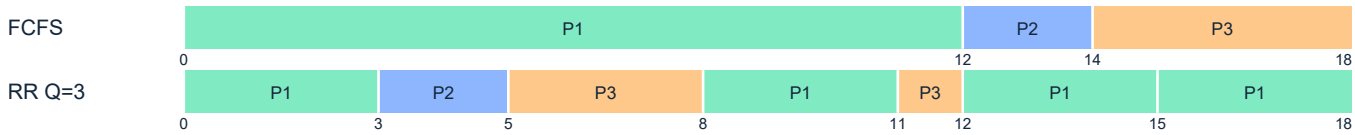
5. Trazas y validación reproducible

Cada bloque identifica al proceso que utiliza la CPU. Los límites son instantes simulados. Gris indica CPU libre. Las trazas se dibujan directamente a partir de los JSON de las ejecuciones visuales.

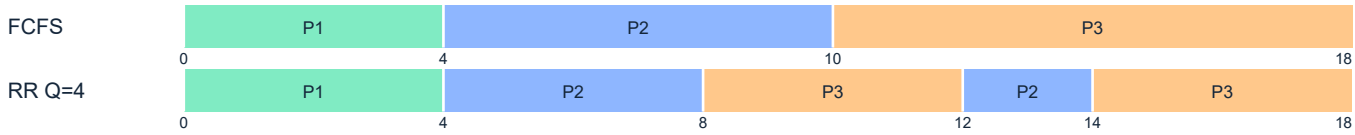
E1 - Carga escalonada



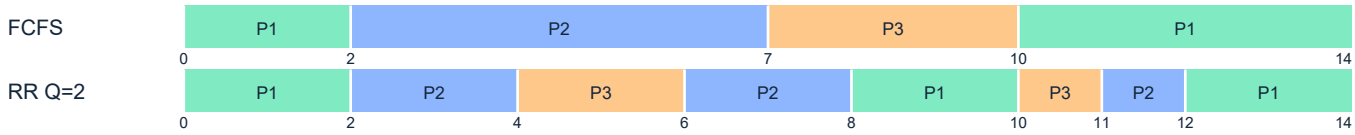
E2 - Efecto convoy



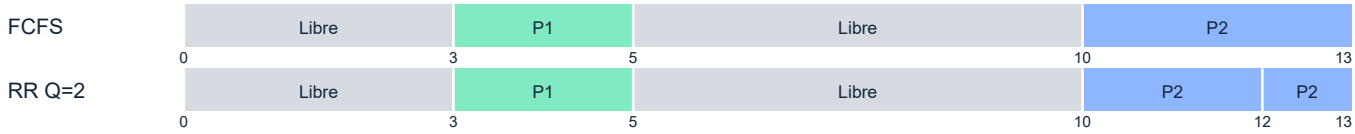
E3 - Llegadas simultáneas



E4 - Bloqueo y reactivación



E5 - CPU ociosa y llegada tardía



Comprobación	Resultado y evidencia
Suite automatizada	31 aprobadas: 19 del simulador y 12 de integración HTTP. Incluye 200 cargas con cuatro configuraciones: 800 comparaciones contra un modelo por ticks independiente.
Consistencia de resultados	Diez ejecuciones visuales coinciden con CSV. Se recalcularon CPU, respuesta, espera y retorno; se verificó continuidad de trazas.
Entradas inválidas	Quantum cero, PID repetido, llegada negativa y CPU fraccionaria rechazados por HTTP 400.
Archivos verificables	escenarios/E1-E5.json; ejecucion/*.csv; pruebas/salida.txt y resultado.json; siete capturas; verificacion.json. Todo dentro de evidencias/.

unittest ejecuta las comprobaciones [3]. Sus entradas, salidas y códigos de terminación se conservan. La interfaz recibió el mismo resultado al ejecutar la suite desde su botón. Aprobar los casos revisados no demuestra ausencia de todos los errores posibles.

6. Riesgos, límites y responsabilidad

Riesgo	Tratamiento y límite
Rendimiento y recursos	La interfaz limita la carga a 30 procesos y 2000 unidades CPU en total. Construye tablas de estados solo al desplegarlas. Quantum pequeño aumenta turnos; no se afirma escalabilidad a millones de eventos.
Estabilidad	Se validan enteros, CPU positiva, llegada no negativa, PID único, quantum positivo y puntos E/S válidos. Los errores se muestran en la interfaz.
Integridad	Copias independientes de entradas, evidencias fechadas y manifiesto SHA-256. Cada nueva corrida se guarda por separado. Regenerar evidencia tras cambiar código o cargas.
Acceso y seguridad	Servidor solo en 127.0.0.1; valida Host y Origin; no sirve archivos arbitrarios. Usar localmente y cerrar al terminar. El repositorio es privado.
Responsabilidad académica	Conservar trazabilidad verificable y explicar los resultados. El registro distingue pruebas guiadas de Esteban y verificación automática.

Límites: una CPU simulada, tiempos enteros y E/S de duración fija sin cola de dispositivo. No incluye RAM, múltiples CPU, prioridades efectivas ni costo temporal de contexto. No mide el SO real. Creación y finalización son estados de objetos simulados, no procesos de Windows.

7. Conclusiones y mejoras

La aplicación permite comparar políticas con las mismas entradas, observar turnos y estados y conservar resultados repetibles. RR ofrece respuesta inicial más temprana en las cargas E1-E4, pero puede aumentar espera y retorno. FCFS y RR consumen el mismo trabajo CPU para una entrada dada. La decisión depende de la carga y la métrica.

Mejoras propuestas: costo configurable de cambio de contexto, cola de dispositivo E/S y evaluación de cargas mayores con límites de recursos. La interfaz gráfica, la edición de procesos y la exportación JSON ya están implementadas.

8. Ejecución, equipo y referencias

Extraer el ZIP y abrir ABRIR_APLICACION.cmd. Requiere Python 3.10+ y su biblioteca estándar; probado con 3.12.14 en Windows. INSTRUCCIONES.md describe la ejecución y DOCUMENTACION_TECNICA.md las funciones. Repositorio privado: https://github.com/Jhomnsito/sistemas-operativos

Esteban Peña participó en ejecución y revisión guiadas, interpretación de escenarios y ensayo de la explicación de validación. TRAZABILIDAD.md registra estas actividades. Se utilizó IA como apoyo en revisión y mejora del código y la interfaz, pruebas y documentación. La entrega se estructura según la consigna del curso [4].

[1] Remzi H. Arpaci-Dusseau y Andrea C. Arpaci-Dusseau. Operating Systems: Three Easy Pieces, cap. 7: Scheduling: Introduction. https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf

[2] Los mismos autores. Operating Systems: Three Easy Pieces, cap. 4: The Abstraction: The Process. https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf

[3] Python Software Foundation. unittest - Unit testing framework. Documentación de Python 3.12. https://docs.python.org/3.12/library/unittest.html

[4] Profesor del curso. Proyecto Final SO 2620.docx, consigna proporcionada: opción 1, requisitos, entrega, sustentación y uso de IA.

Fuentes web originales verificadas el 4 de noviembre de 2026. Registro: evidencias/fuentes.json. La referencia [4] es el documento del curso proporcionado por el estudiante, sin URL pública.